

SpendWise

Personal Finance Tracker — Project Brief

Project Name	SpendWise — Personal Finance Tracker
Type	Full-Stack Web Application
Stack	Flask (Python) + React (JSX) + PostgreSQL
External API	Frankfurter API (live currency conversion, no key required)
Estimated Effort	30–38 hours across 6 phases
Deployment	Render (frontend + backend)

1. Problem & Purpose

Most people have no clear picture of where their money goes each month. They rely on mental estimates or scattered notes, and by the time they realise they have overspent in a category, it is too late to adjust. Generic spreadsheet tools require manual setup, offer no feedback, and do not flag when spending exceeds a set budget.

Target Users

Young professionals and students who earn income, pay recurring expenses, and want to build better financial habits — but lack the time or tools to do so. Their core frustration: checking their bank account and feeling surprised because they have no running awareness of their spending.

Solution

SpendWise lets users log income and expenses, organise them by category, set monthly budgets per category, and receive instant feedback on budget health. The backend handles all analytical work — calculating totals, comparing actuals to budgets, flagging overspending — while the frontend presents results in a clean, functional interface.

2. Core Features

Authentication

- User registration with hashed passwords (Flask-Bcrypt)
- JWT-based login and protected routes (Flask-JWT-Extended)
- All financial data is scoped to the authenticated user

Transaction Management (full CRUD)

- Log income or expense transactions with amount, date, description, and category

- Edit or delete existing transactions
- Foreign currency support — amounts converted to USD via Frankfurter API

Category Management (full CRUD)

- Create, edit, and delete custom spending categories (e.g. Food, Rent, Travel)
- Categories belong to the user and link to transactions and budgets
- Filter transactions by month or category

Budget Manager

- Set a monthly spending limit per category
- Update or remove budget limits at any time

Backend Analytics

- Calculate total income, total expenses, and running balance per user per month
- Compare actual spending vs. budget limit for each category
- Flag categories where the user has exceeded their budget

Summary Dashboard

- Aggregated financial totals from the /summary endpoint
- Budget health per category (on track vs. over budget)
- Clean, functional UI built with Material UI

3. Data Models

Model	Key Fields	Relationships
User	id, username, email, password_hash	has many Transactions, Budgets
Category	id, name, user_id	has many Transactions, Budgets
Transaction	id, amount, currency, amount_usd, type, date, description, user_id, category_id	belongs to User, Category
Budget	id, amount, month, year, user_id, category_id	belongs to User, Category

4. API Endpoints

Endpoint	Purpose
<code>POST /signup</code>	Create a new user account and return JWT token

POST /login	Authenticate and return JWT token
GET/POST /transactions	Retrieve all or create a transaction
GET/PATCH/DELETE /transactions/<id>	Read, update, or delete a specific transaction
GET/POST /categories	Retrieve all or create a category
GET/PATCH/DELETE /categories/<id>	Read, update, or delete a specific category
GET/POST /budgets	Retrieve all or create a budget
GET/PATCH/DELETE /budgets/<id>	Read, update, or delete a specific budget
GET /summary	Aggregated totals: income, expenses, balance, budget vs. actual
GET /exchange-rate/<currency>	Fetch live exchange rate and return converted USD amount

5. Frontend Components

React Component Structure

- App.jsx — Router setup, JWT token context
- Login.jsx / Signup.jsx — Authentication forms
- Dashboard.jsx — Summary view: totals and budget health per category
- TransactionList.jsx — Table of all transactions with edit/delete controls
- AddTransactionForm.jsx / EditTransactionForm.jsx — Add/edit forms with currency selector
- BudgetManager.jsx — Set and update monthly budgets per category
- CategoryManager.jsx — Full CRUD interface for categories

6. Technology Stack

Layer	Technologies
Backend	Flask, Flask-JWT-Extended, Flask-Bcrypt, SQLAlchemy, Marshmallow, PostgreSQL
Frontend	React, React Router, useState, useEffect, useContext, Material UI
External API	Frankfurter API (free, no key required, live exchange rates)
Dev Tools	Postman / cURL, Visual Studio Code, Git / GitHub, Render

7. Project Timeline

Phase	Deliverable	Est. Hours
-------	-------------	------------

Planning & Research	Completed project architecture plan	4–5 hrs
Backend Setup & DB Modeling	Functional REST API with auth and database	8–10 hrs
Business Logic & Analytics	Working analytics endpoints and validated backend logic	5–6 hrs
Frontend Development	Functional frontend connected to backend services	6–8 hrs
Testing, Debugging & Deployment	Stable deployed application ready for presentation	4–5 hrs
Documentation & Presentation	Final project submission materials	3–4 hrs
Total Estimated Effort		30–38 hrs

8. Risks & Open Questions

External API Reliability

Frankfurter API is free and requires no authentication. The application will include graceful error handling if the API is temporarily unavailable or returns unexpected data.

JWT Token Expiration

If authentication sessions expire too quickly during testing or demonstration, refresh token support may need to be implemented using Flask-JWT-Extended.

Currency Precision & Rounding

Financial calculations require consistent rounding. The application will use Python's decimal module to avoid floating-point precision issues.

Frontend/Backend Integration

Managing JWT authentication between React and Flask may require additional debugging for protected routes and token persistence.

Scope Management

Additional features such as charts or advanced analytics may be deferred if core CRUD functionality and authentication require more implementation time than estimated.